

Abstract State Machines (parte 2): regole di transizione



Elvinia Riccobene
elvinia.riccobene@unimi.it

Regole di transizione ASM

In matematica le *algebre* sono *statiche* : non cambiano col passare del tempo.

In Informatica, gli *stati* sono *dinamici* : evolvono essendo aggiornati durante le computazioni.

Aggiornare stati astratti (*abstract states*) significa cambiare l'interpretazione delle (o solo di alcune) funzioni *dinamiche* della segnatura della macchina.

Regole di transizione ASM

Il modo in cui una macchina ASM aggiorna il proprio stato è descritto da **regole di transizione** (**transitions rules**) di una certa “forma”

L'insieme delle regole di transizione di una ASM definiscono la sintassi di un **programma ASM**

Sia Σ un vocabolario. Le *regole di transizione di una ASM* sono espressioni sintattiche su Σ generate come segue attraverso l'uso di **costruttori di regole**

Costruttore base: update rule

- *Update Rule:*

$$f(t_1, \dots, t_n) := s$$

dove:

- f è un nome di funzione **dinamica** n -aria di Σ
- $t_1, \dots, t_n$ e s sono termini di Σ

Significato: Nello stato successivo, il valore di f per gli argomenti $t_1, \dots, t_n$ è aggiornato a s .

Se f è 0-aria, cioè una variabile x , l'aggiornamento ha la forma **$x := s$**

Costruttore base: update rule

- *Update Rule:*

Esempio

DisplayTime := CurrTime

con **DisplayTime: Real** variabile (funzione 0-aria) reale

cnt_state:= Opening

L'update rule è il costrutto di base delle regole di transizione e determina l'aggiornamento di una **locazione** ASM
Vediamo il significato e la definizione...

Locazione ASM

- Nelle ASM gli stati sono associati a un insieme di **valori** di qualsiasi tipo, memorizzate in apposite **locazioni**
- Matematicamente una **locazione** è definita come la *coppia* $(f, (v_1, \dots, v_n))$ con **f** nome di funzione e $(v_1, \dots, v_n)$ i suoi argomenti
- In uno stato S , una **locazione** ha un **valore**:
 - Funzione può essere:
 - **Variabili (0-arie)** $x, y, \dots$
 - **Funzioni (n-arie)** $f(1), \text{name}(2), \dots$

Aggiornamento di locazione

- Le transizioni di stato delle ASM, tramite l'*update rule*, causano **aggiornamenti** dei valori contenuti nelle locazioni
- Se $loc = (f, (v_1, \dots, v_n))$ ed $valore(loc) = a$, l'**aggiornamento** (o **update**) di loc ha la forma

$f(v_1, \dots, v_n) := newval$

L'aggiornamento cambia il valore di loc , e quindi di $f(v_1, \dots, v_n)$, da a a $newval$

Costruttore if-then-else

- *Conditional Rule:*

if φ then R else S

Significato: se φ è vera, allora esegui R , altrimenti esegui S

Esempio

**if CurrTime = DisplayTime + Delta
then DisplayTime := CurrTime**

con DisplayTime, CurrTime, e Delta variabili (funzione 0-arie)
reali

Semplice esempio: CLOCK

- Modellare il funzionamento di un CLOCK che aggiorna il **display** del tempo ad ogni **Delta** intervallo di tempo.
- Vocabolario Σ
 - **DisplayTime**: Real
 - orario visualizzato sul display
 - **Delta**: Real
 - intervallo di tempo
 - **CurrTime**: Real
 - tempo corrente
 - **+** : Real x Real $\rightarrow$ Real (addizione)

Semplice esempio: CLOCK

Vocabolario Σ

- **CurrTime**: Real
- **DisplayTime**: Real
- **Delta**: Real
- **+** : Real x Real $\rightarrow$ Real (addizione)

Regola di transizione (**main rule**):

if CurrTime = DisplayTime + Delta
then DisplayTime := CurrTime

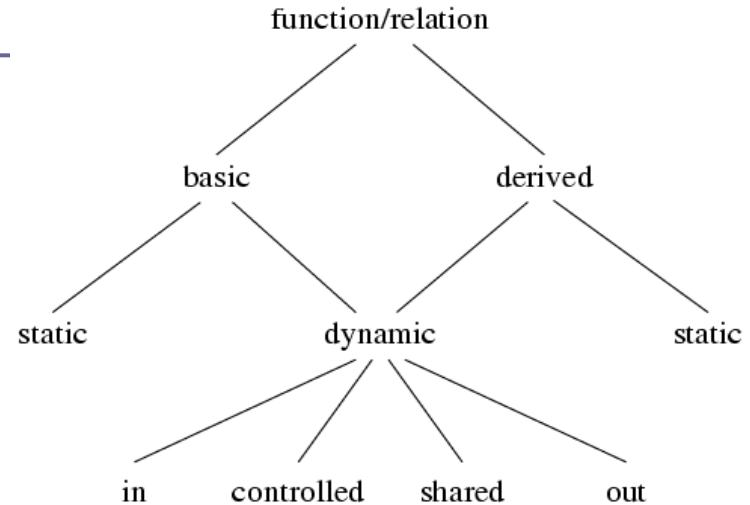
Classificazione funzioni

- Unica regola per il cambiamento di stato:
 - **if** $\text{CurrTime} = \text{DisplayTime} + \text{Delta}$
then $\text{DisplayTime} := \text{CurrTime}$

- Analizziamo l'aggiornamento delle funzioni utilizzate:
 - **DisplayTime**: il valore viene aggiornato dalla regola
 - **Delta**: valore sempre fisso
 - **CurrTime**: aggiornato da chi?
 - **+**: somma, la sua legge è sempre fissa

Classificazione delle funzioni ASM

Sia ***M*** un ASM e
env l'ambiente di ***M***



Dynamic: i valori dipendono dagli stati di ***M***

- **in** (**monitored**): lette (non aggiornate) da ***M***, scritte da ***env***
- **out**: scritte (ma non lette) da ***M***, lette da ***env***
- **controlled**: lette e scritte da ***M***
- **shared**: lette e scritte da ***M*** e da ***env***

Derived: valori computati da funzioni monitorate e funzioni statiche per mezzo di una "legge" o "schema" fissati a priori

Classificazione di funzioni del CLOCK

- **Dinamica monitored: CurrTime:** Real
(viene incrementato dall'esterno)
- **Dinamica controlled: DisplayTime:** Real
(viene incrementata dalla regola)
- **Static: Delta:** Real
(non cambia al variare dello stato)
 $+ : \text{Real} \times \text{Real} \rightarrow \text{Real}$ (funzione con legge fissa)
- **Static: $+$:** Real $\times$ Real $\rightarrow$ Real
(funzione con legge fissa)

Regola di transizione (main rule):

if CurrTime = DisplayTime + Delta
then DisplayTime := CurrTime

Con la seguente funzione **derived**:

ClockTick = 1 iff CurrTime = DisplayTime + Delta

che esprime una procedura di computazione standard, la regola diventa
if ClockTick = 1 then DisplayTime := CurrTime

Tutti i costruttori di regole

Nome	Significato	Sintassi
<i>Skip rule</i>	Non fa nulla	skip
<i>Update rule</i>	Aggiorna il valore di f per gli argomenti $(s_1, \dots, s_n)$ a t Nota: f è un nome di funzione dinamica	$f(s_1, \dots, s_n) := t$
<i>Block rule</i>	P e Q sono eseguite in parallelo	par P Q endpar
<i>Sequence rule</i>	P e Q sono eseguite in sequenza	seq P Q endseq
<i>Conditional rule</i>	Se φ è vera, esegui P , altrimenti Q	if φ then P else Q
<i>Let rule</i>	Assegna il valore di t a x e quindi esegui P	let $x = t$ in P
<i>Forall rule</i>	Esegui P in parallelo per ogni x che soddisfa φ	forall x with φ do P
<i>Choose rule</i>	Scegli un x che soddisfa φ e quindi esegui P	choose x with φ do P
<i>MacroCall rule</i>	Esegui la regola di transizione r con parametri $t_1, \dots, t_n$	$r(t_1, \dots, t_n)$

Costruttori skip e let

- *Skip Rule:*

skip

Significato: non fare niente

if $x > 0$ **then** $x := x + 1$

else skip

- *Let Rule (se R ha parametri, realizza il passaggio per valore):*

let $x = t$ **in** $R(x)$

Significato: Assegna il valore di t a x ed esegui R

let $x = f_1(f_2(f_3(y)))$ **in**

$g(x) := 7$

Costruttore “par” per block rule

Block Rule (composizione parallela):

```
par
  R
  S
endpar
```

Modella **synchronous bounded parallelism**

- **bounded parallelism** = simultaneous application of different functions

Significato: *R e S sono eseguite in parallelo*

Esempio 1: due aggiornamenti paralleli, ipotesi $f(5)=1$

$f(5) := 2$

$g(3) := f(5)$ -- allo stato successivo $g(3)=1$

con $f:\text{Integer} \rightarrow \text{Integer}$

Esempio 2: due aggiornamenti paralleli di due valori della stessa funzione

$f(5) := 2$

$f(3) := 7$

con $f:\text{Integer} \rightarrow \text{Integer}$

Aggiornamenti Consistenti

- A causa del parallelismo (regole Block e Forall), una regola di transizione può richiedere *più volte l'aggiornamento di una stessa funzione* per gli stessi argomenti
- si richiede in tal caso che tali aggiornamenti *siano consistenti*.

Aggiornamenti e locazioni

Ricordiamo che...

data la funzione $f(a_1, \dots, a_n)$ in uno stato S_i della macchina:

- La coppia $\text{loc} = (f, (a_1, \dots, a_n))$ è detta *locazione* e rappresenta matematicamente il valore di $f(a_1, \dots, a_n)$ in memoria

- Un *aggiornamento* (o *update*) è la coppia

$$(\text{loc}, b) = ((f, (a_1, \dots, a_n)), b)$$

Significato: l'interpretazione della funzione f in S_i viene modificata per gli argomenti $a_1, \dots, a_n$ con il valore b in S_{i+1}

- Un *update set* è un insieme di aggiornamenti

Aggiornamenti Consistenti

DEF: Un update set U è *consistente*, se vale la condizione:

Per ogni locazione $(f, (a_1, \dots, a_n))$,

se $((f, (a_1, \dots, a_n)), b) \in U$ ed

$((f, (a_1, \dots, a_n)), c) \in U$, allora $b = c$

Se $b \neq c$, allora l'update si dice *inconsistente*

Aggiornamenti Consistenti

- Se l'update set U è consistente, allora i suoi **aggiornamenti** possono essere effettivamente eseguiti (**fired**) in un dato stato.
Il risultato è un nuovo stato (di arrivo) dove le **interpretazioni** dei nomi **delle funzioni dinamiche** sono cambiati secondo U .
- Le **interpretazioni** dei nomi **delle funzioni statiche** sono gli stessi dello stato precedente (di partenza).
- Le **interpretazioni** dei nomi **delle funzioni monitorate** sono date dall'ambiente esterno e possono dunque cambiare in maniera arbitraria.

Rule constructors

(Macro) Call Rule:

$r [t_1, \dots, t_n]$

Significato: Chiama r (*regola con parametri*) con argomenti $t_1, \dots, t_n$

Una *definizione di regola per un nome di regola r* di arietà n è un'espressione

$$r (x_1, \dots, x_n) = R$$

dove R è una regola di transizione.

In una call rule $r [t_1, \dots, t_n]$ le variabili x_i che occorrono nel corpo R della definizione di r vengono sostituite dai parametri t_i (*modularità*)

Esempio: Advanced Clock

- Un CLOCK che ad ogni passo incrementa i secondi
- signature:
 - domain Second
 - domain Minute
 - domain Hour
 - controlled seconds : Second
 - controlled minutes : Minute
 - controlled hours : Hour

Esempio: Advanced Clock

□ regole

macro rule **IncMinHours** =

par

if minutes = 59 then

hours := (hours + 1) mod 24

minutes := (minutes + 1) mod 60

main rule **Main** =

par

if seconds = 59 then

IncMinHours[]

seconds := (seconds + 1) mod 60

Esempio: Advanced Clock

□ Stato iniziale

default init s0:

function seconds = 0

function minutes = 0

function hours = 0

Esempio: Advanced Clock

- Passare dalla notazione matematica alla notazione concreta può richiedere alcune precisazioni:
- signature:
 - domain **Second** subsetof Integer
domain Second = {0..59}
 - domain **Minute** subsetof Integer
domain Minute = {0..59}
 - domain **Hour** subsetof Integer
domain Hour = {0..23}
 - controlled **seconds** :Second
 - controlled **minutes** :Minute
 - controlled **hours** :Hour

Advanced Clock 2

- Incrementa l'ora (mod 3) all'arrivo di un segnale
- Stessa segnatura di prima ma:
 - domain Second={0..3}
 - domain Minute={0..3}
 - domain Hour={0..3}
- Inoltre ...
 - monitored **signal**:Boolean
 - Inizializzata a *false*
 - default init s1:
 - function signal = false

Advanced Clock 2 : Regole

macro rule **IncMinHours** =

par

if minutes=2 **then**

hours := (hours+ 1) mod 3

minutes := (minutes + 1) mod 3

main rule **AdvancedClock** =

if signal **then**

par

if seconds=2 **then**

IncMinHours[]

seconds := (seconds+1) mod 3

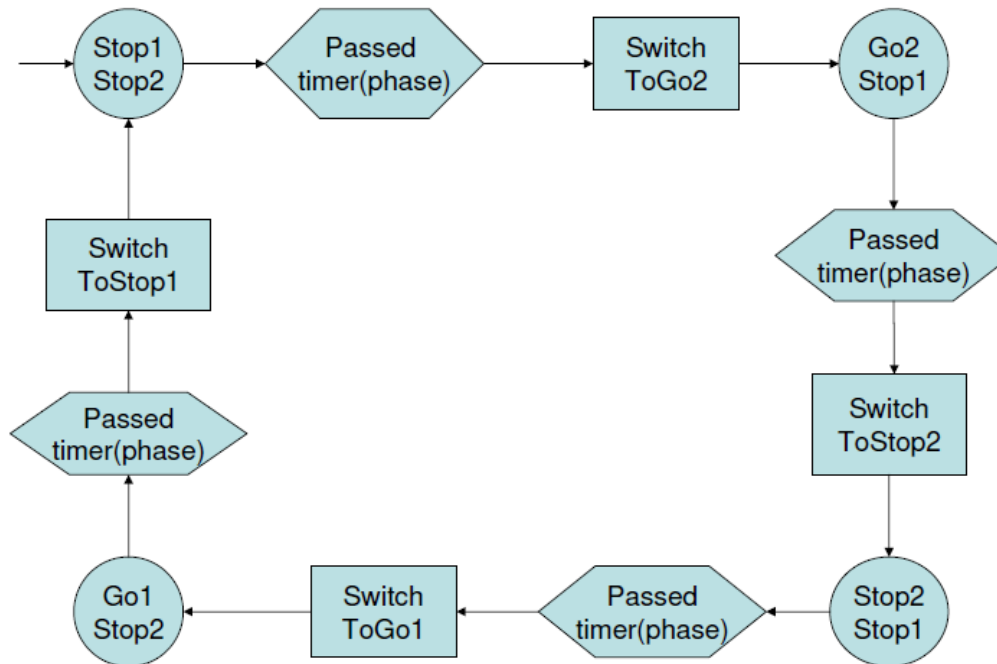
One way traffic light control system (Reqs.)

- un sistema costituito da una coppia di semafori (LIGHTUNIT1 e LIGHTUNIT2) posti alle estremità di una via a senso unico alternato, collegati a un computer che li controlla
- Ogni semaforo è dotato di una luce di *Stop* (luce rossa) ed una luce di *Go* (luce verde)
- Il computer accende/spegne le luci inviando ai semafori i segnali *Rpulses* e *Gpulses*

One way traffic light control system (Reqs.)

- Lo stato delle luci dei due semafori varia seguendo un ciclo in quattro fasi:
 - per 50 secondi tutte e due i semafori mostrano il segnale di Stop
 - per 120 secondi il semaforo LIGHTUNIT1 mostra il segnale di Stop e LIGHTUNIT2 il segnale di Go
 - per 50 secondi tutti e due i semafori mostrano nuovamente il segnale di Stop
 - per 120 secondi il semaforo LIGHTUNIT2 mostra il segnale di Stop e LIGHTUNIT1 il segnale di Go

One way traffic light control system (control state ASM)



One way traffic light control system (Spec.)

Segnatura :

- enum domain **LightUnit** = {LIGHTUNIT1 | LIGHTUNIT2}
- enum domain **PhaseDomain** = { STOP1STOP2 | GO2STOP1 | STOP2STOP1 | GO1STOP2 }
- domain **Intervals** subsetof Integer
 - domain Intervals = {50 , 120}
 - Rappresenta i periodi di durata dei segnali di stop e di go
- dynamic controlled **phase** : PhaseDomain
- dynamic controlled **stopLight** : LightUnit -> Boolean
- dynamic controlled **goLight** : LightUnit -> Boolean
 - rappresentano le luci di stop e go del semaforo
- dynamic monitored **passed** : Intervals -> Boolean

One way traffic light control system (Spec.)

main rule **main** =

par

stop1stop2_to_go2stop1 []

go2stop1_to_stop2stop1 []

stop2stop1_to_go1stop2 []

go1stop2_to_stop1stop2 []

endpar

One way traffic light control system (Spec.)

```
rule stop1stop2_to_go2stop1 =  
  If (phase = STOP1STOP2 and passed (50))  
    then  
      par  
        switchLightUnit [ LIGHTUNIT2 ]  
        phase := GO2STOP1  
      endpar  
    endif
```

One way traffic light control system (Spec.)

```
rule switchLightUnit (l in LightUnit ) =  
  par  
    switch [ goLight (l)]  
    switch [ stopLight (l)]  
  endpar
```

```
macro rule switch (l in Boolean ) =  
  l := not(l)
```

One way traffic light control system (Spec.)

```
rule stop1stop2_to_go2stop1 =  
  if( phase = STOP1STOP2 and passed (50) )  
    then  
      par  
        switchLightUnit [ LIGHTUNIT2 ]  
        phase := GO2STOP1  
      endpar  
    endif
```

One way traffic light control system (Spec.)

```
rule go2stop1_to_stop2stop1 =  
  if( phase = GO2STOP1 and passed (120) )  
  then  
    par  
      switchLightUnit [ LIGHTUNIT2 ]  
      phase := STOP2STOP1  
    endpar  
  endif
```

One way traffic light control system (Spec.)

```
rule stop2stop1_to_go1stop2 =  
  if( phase = STOP2STOP1 and passed (50) )  
  then  
    par  
      switchLightUnit [ LIGHTUNIT1 ]  
      phase := GO1STOP2  
    endpar  
  endif
```

One way traffic light control system (Spec.)

```
rule go1stop2_to_stop1stop2 =  
  if( phase = GO1STOP2 and passed (120) )  
  then  
    par  
      switchLightUnit [ LIGHTUNIT1 ]  
      phase := STOP1STOP2  
    endpar  
  endif
```

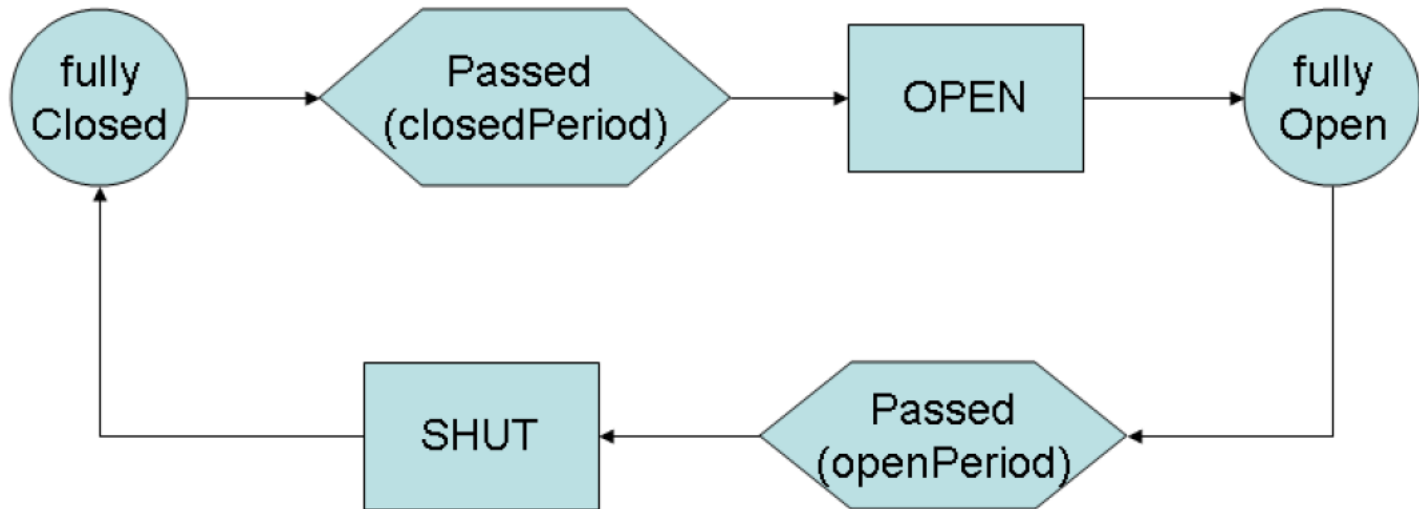
Esempio: SluiceGate (Reqs) 1/2

- sistema di irrigazione che controlla il flusso dell'acqua tramite una chiusa
- la chiusa è sbarrata da una saracinesca che si può alzare ed abbassare
- La saracinesca deve essere totalmente aperta per 10 minuti ogni tre ore; nel resto del tempo deve essere totalmente chiusa
 - dei sensori segnalano se è completamente chiusa o completamente aperta
- La saracinesca viene aperta/chiusa ruotando delle viti che sono manovrate da un motore

Esempio: SluiceGate (Reqs) 2/2

- Al motore vengono inviati 4 segnali:
 - per ruotare le viti in senso orario,
 - per ruotarle in senso antiorario,
 - per accenderlo e
 - per spegnerlo
- Il computer controlla il sistema emettendo i quattro segnali per guidare il motore;
- il computer è collegato ai due sensori posti alla estremità del percorso della saracinesca per conoscerne la posizione

Esempio: SluiceGate (Spec.)



Segnatura

- domain **Minutes** subsetof Integer
- enum domain **PhaseDomain** =
 { FULLYCLOSED | FULLYOPENED }
- dynamic controlled **phase** : PhaseDomain
- dynamic monitored **passed** : Minutes -> Bool
- domain **Minutes** = {10 , 170}
 - **passed(10)** e **passed(170)** indicano se è trascorso l'intervallo di tempo in cui la saracinesca deve essere completamente aperta e (risp.) completamente chiusa

Specifica ASM

main rule **main** =

par

if(phase = FULLYCLOSED) then

if(passed (170)) then

par

open []

phase := FULLYOPENED

if(phase = FULLYOPENED) then

if(passed (10)) then

par

shut []

phase := FULLYCLOSED

Per simulare il
modello:

rule open = skip
rule shut = skip

*Restano da
raffinare le
regole **open**
e **shut***

Costruttore seq

- *Seq Rule (composizione sequenziale):*

seq R S endseq

Significato: *R e S sono eseguite in sequenza (gli update causati in R sono già visibili in S; lo stato tra R ed S non è visibile)*

Esempio: due aggiornamenti in sequenza dello stesso valore della funzione, ipotesi $f(5)=1$

seq

$f(5) := 2$

$f(3) := f(5)$ -- allo stato successivo $f(3)=2$

con $f: \text{Integer} \rightarrow \text{Integer}$

Esempio: fattoriale

- dynamic monitored **valore**: Integer
- dynamic controlled **indice**: Integer
- dynamic controlled **fattoriale**: Integer

- macro rule **fattor** =
 - if (indice>1) then
 - par
 - fattoriale := fattoriale*indice
 - indice := indice-1
 - endpar
- endif

Esempio: fattoriale

main rule **main** =

seq

if (indice=1) then

if (valore>0) then

par

indice := valore

fattoriale := 1

endif

fattor[]

endseq

Inizializzazione:

default init s0:

function indice = 1



Esempio: fattoriale con stop

- dynamic monitored **valore**: Integer
- dynamic controlled **indice**: Integer
- dynamic controlled **fattoriale**: Integer
- **dynamic controlled stop**: Boolean
- macro rule **fattoriale** =
 if (indice>1) then
 seq
 fattoriale := fattoriale*indice
 indice := indice-1
 if indice =1 then stop:=true
 endseq
 endif

Esempio: fattoriale con stop

main rule **main** =

if stop = false

then

seq

if (indice=1) then

if (valore>0) then

par

indice := valore

fattoriale := 1

endif

fattoriale[]

endseq

Inizializzazione:

default init s0:

function indice = 1

function stop = false

Costruttori forall e choose

- *Forall Rule:*

forall x **with** φ **do** $R[x]$

Significato: Esegui R in parallelo per ogni x che soddisfa φ

Implementa il concetto di **parallelismo sincrono** (*potentially unbounded*)

- *Choose Rule:*

choose x **with** φ **do** $R[x]$

Significato: Esegui R per un x *scelto random* che soddisfi φ

Implementa il concetto di **non-determinismo**

Esempio: ordinare un vettore

macro rule **swap**(\$x in Integer, \$y in Integer) =

par

\$x := \$y

\$y := \$x

endpar

macro rule **swapSort** =

choose \$i in {0..9}, \$j in {0..9} with (\$i < \$j and
vect(\$i) > vect(\$j)) do

swap[vect(\$i), vect(\$j)]

main rule **r_main** =

swapSort[]

Nota! Non occorrono variabili di appoggio per lo swap, perché i nuovi valori degli aggiornamenti saranno disponibili solo nello stato successivo!

Esecuzione

- In s_0 : Vett = [2,5, 4,1,17,3]
 - i, j variano da 0 a 5
 - choose seleziona: i=0, j=3
- In s_1 : Vett = [1,5,4,2,17,3]
 - choose seleziona: i=1, j=5
- In s_2 : Vett = [1,3,4,2,17,5]
 - choose seleziona: i=1, j=3
- In s_3 : Vett = [1,2,4,3,17,5]
 - choose seleziona: i=2, j=3
- In s_4 : Vett = [1,2,3,4,17,5]
 - choose seleziona: i= 4, j=5
- In s_5 : Vett =[1,2,3,4,5,17]

Esempio su aggiornamenti inconsistenti

Specifica che può causare inconsistent updates
a causa del non determinismo della choose
rule

abstract domain **Orders**

enum domain **Status** = {PENDING | INVOICED | CANCEL}

static o1: Orders

static o2: Orders

static o3: Orders

controlled orderStatus: Orders -> Status

Esempio su aggiornamenti inconsistenti

main rule **main** =

par

choose \$o in Orders with true do

orderStatus(\$o) := INVOICED

choose \$oo in Orders with true do

orderStatus(\$oo) := CANCEL

endpar

default init s0:

function orderStatus(\$o in Orders) = PENDING

ASMs: riassumendo...

Una *Abstract State Machine* M è una terna (Σ, A, R)

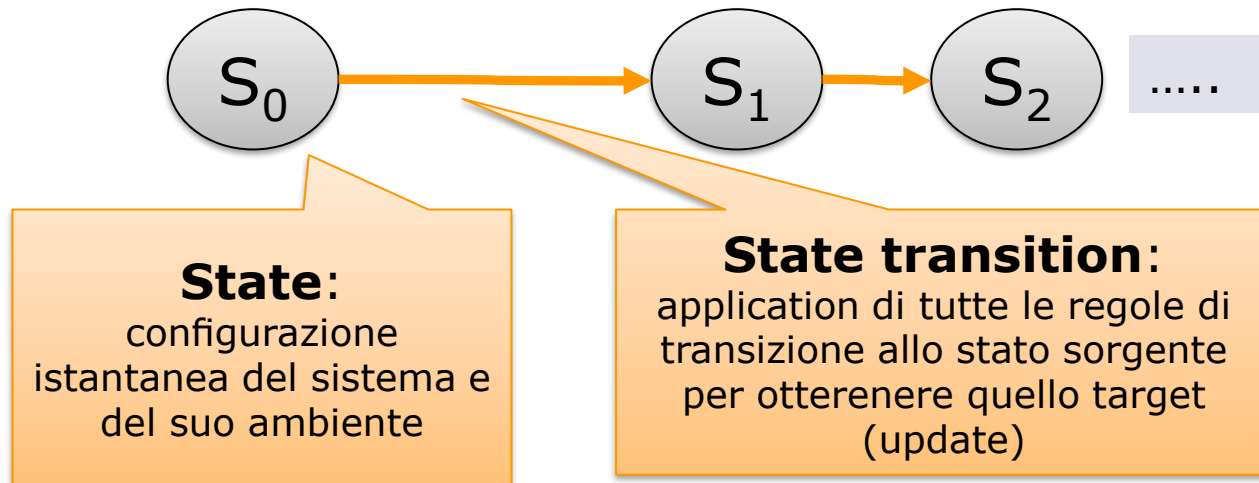
- un vocabolario Σ ,
- uno stato iniziale A per Σ ,
- un insieme R di nomi di regole con
 - un nome di regola di arità zero, la cosiddetta *main rule* (l' "entry point" per l'esecuzione della macchina)
 - una definizione di regola per ogni nome di regola

La *semantica delle regole di transizione* è data dall'insieme di tutti gli aggiornamenti.

Eeguire una ASM

La **run** (o esecuzione) di una ASM M è definita come:

- Una sequenza finita o infinita $S_0, S_1, \dots, S_n$ di stati di M , dove S_0 è lo stato iniziale e ciascun stato S_{n+1} è ottenuto dal precedente S_n eseguendo simultaneamente tutte le regole di transizione che sono *eseguibili* in S_n



La Riserva di una ASM

Extend Rule:

Per estendere un sub-universo del superuniverso con nuovi elementi si usa la notazione:

import x **do** $R[x]$

Il **significato** di tale costrutto è: sceglie un elemento x da *Reserve* (dominio predefinito di simboli “freschi”), lo cancella da *Reserve*, aggiunge x al superuniverso ed esegue R su x

- Spesso si preferisce la forma seguente:

extend U **with** x **do** $R[x]$

come abbreviazione per **import** x **do**

$U(x) := \text{true}$ **seq** $R[x]$

Es: operazioni sullo STACK

Signature:

domain POS

domain ELEM

controlled next: POS $\rightarrow$ POS

controlled head: POS

controlled stack: POS $\rightarrow$ ELEM

ADD(Elem) =

extend POS **with** pos **do**

$R[pos]$ {
next(pos) := head
head := pos
stack(pos) := Elem

REMOVE =

if head $\neq$ undef

then head := next(head)

else skip